AI ASSISTANT CODING
ASSIGNMENT - 20.3

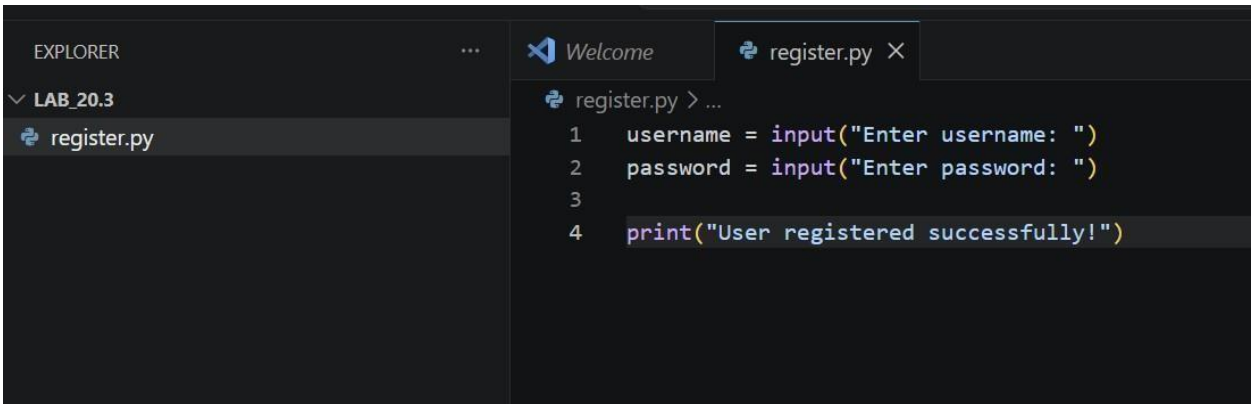
Name: Saketh Birru
Ht.no: 2303A51403
Batch: 06

Task1: Password Strength & Validation Check

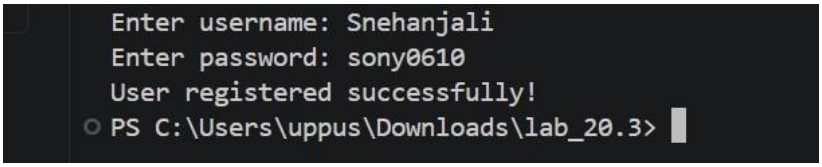
Prompt: Generate Python user registration program

Code:

AI-Generated Code:

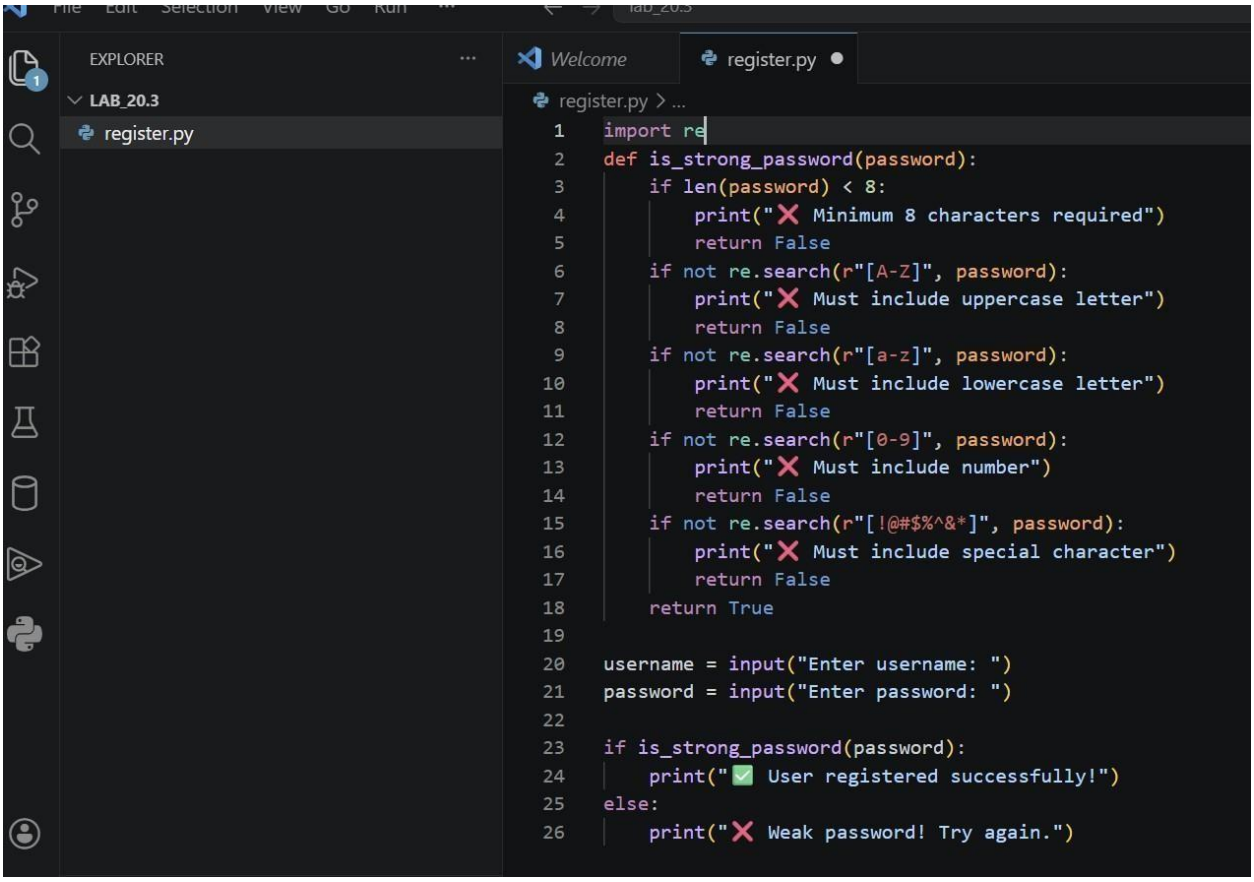


Output:

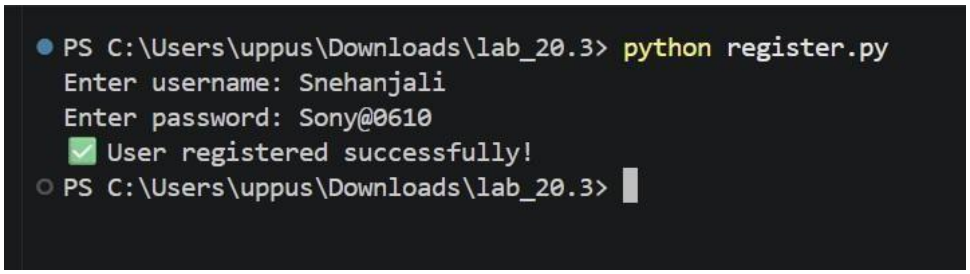


- The program accepts passwords without any validation checks.
- Weak passwords are allowed, making the system insecure.

Corrected Code:



Output:



Explanation:

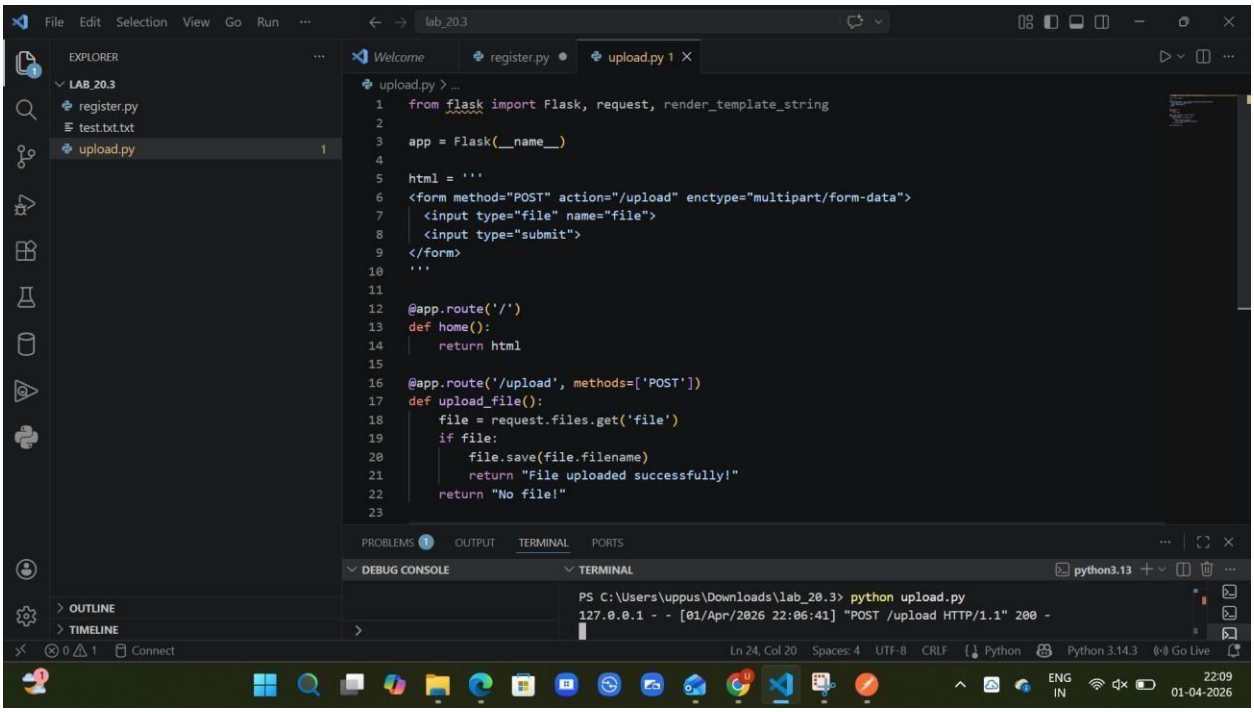
The initial program does not check the strength of the password. It accepts any password entered by the user, including weak ones. This makes the system insecure and vulnerable to attacks.

Task2: Insecure File Upload Handling

Prompt:

Generate a Flask-based file upload program that allows users to upload files.

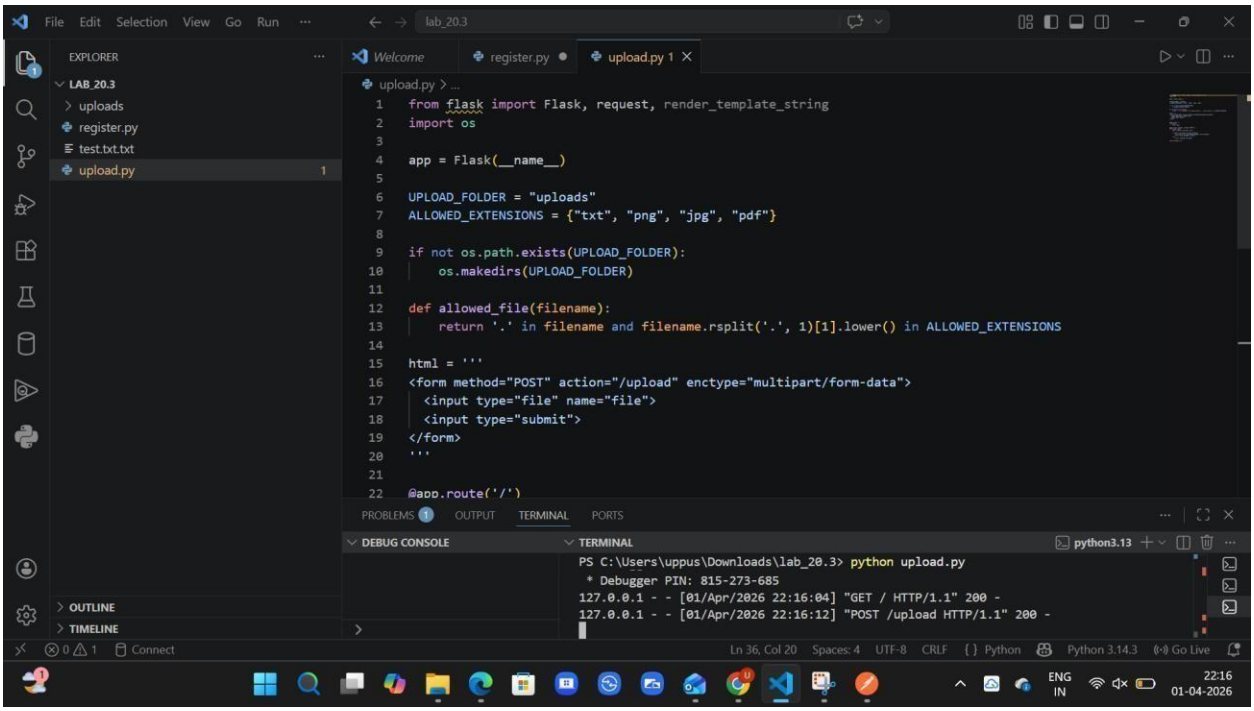
Code:Ai Generated Code



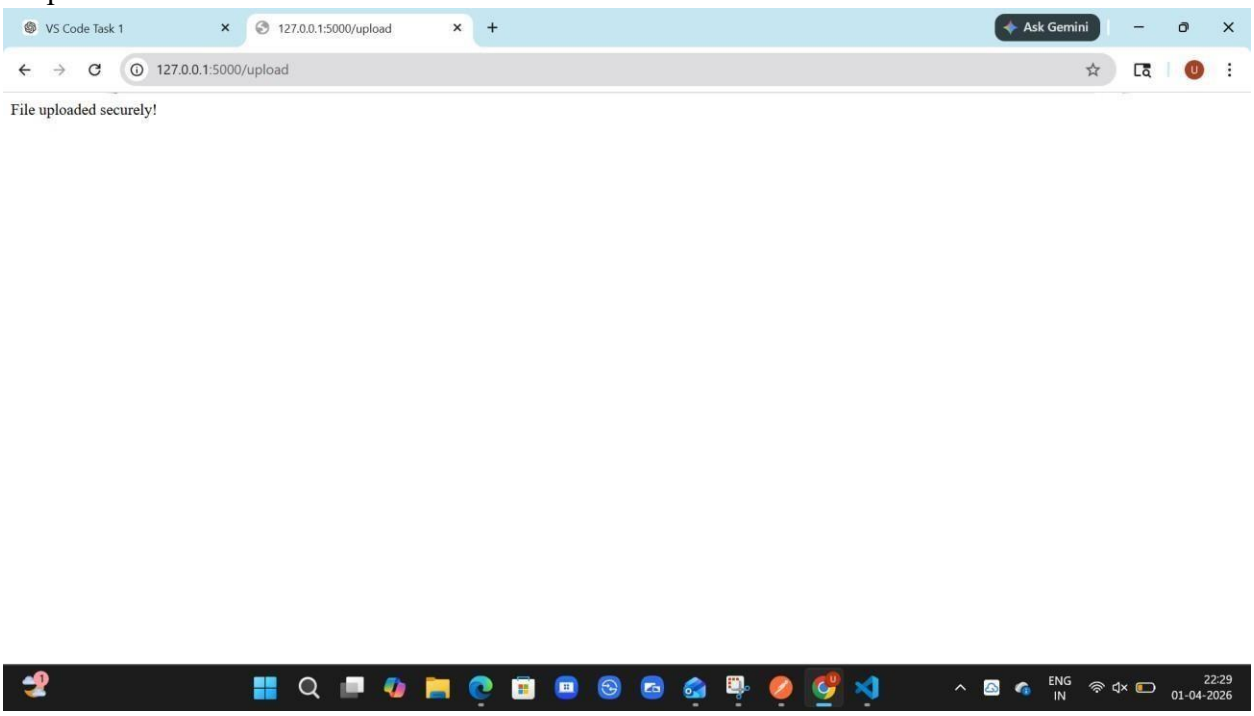
Output:

- The program allows uploading any type of file
- No validation of file extension

Corrected code:



Output:



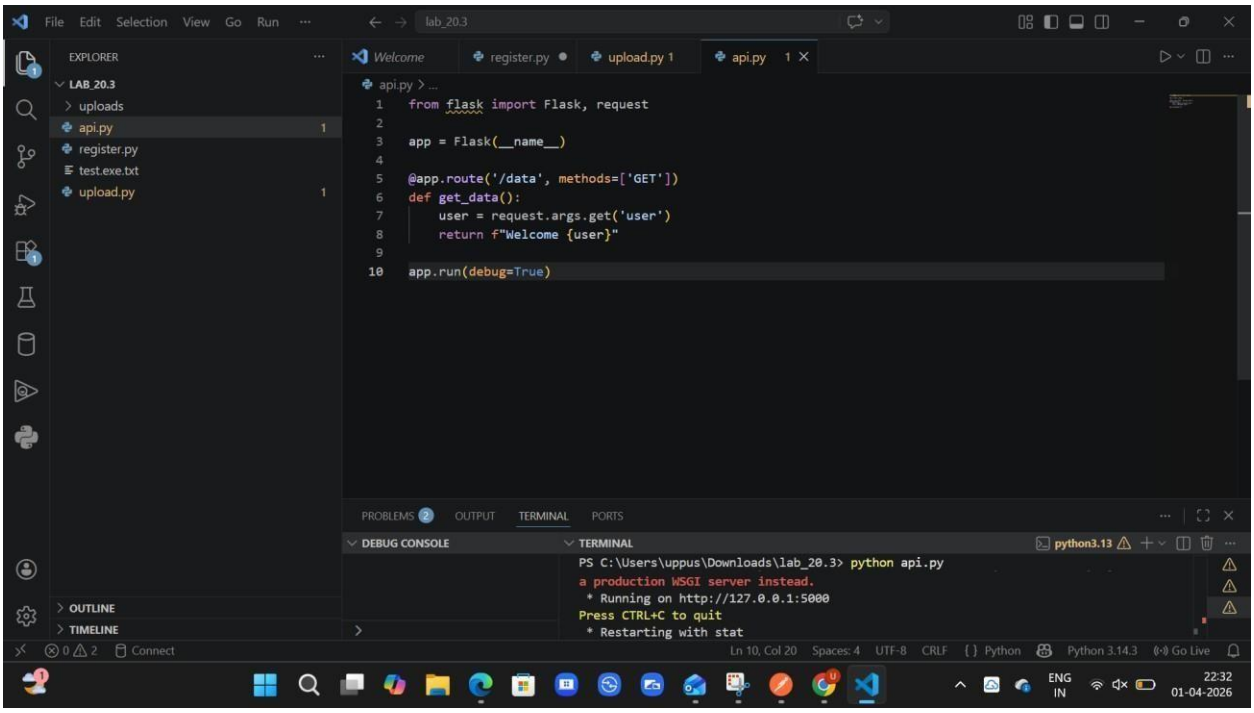
Explanation:

- The initial code allows uploading any file without checking its type, making it insecure.
- Malicious files like `.exe` can be uploaded, which may harm the system.

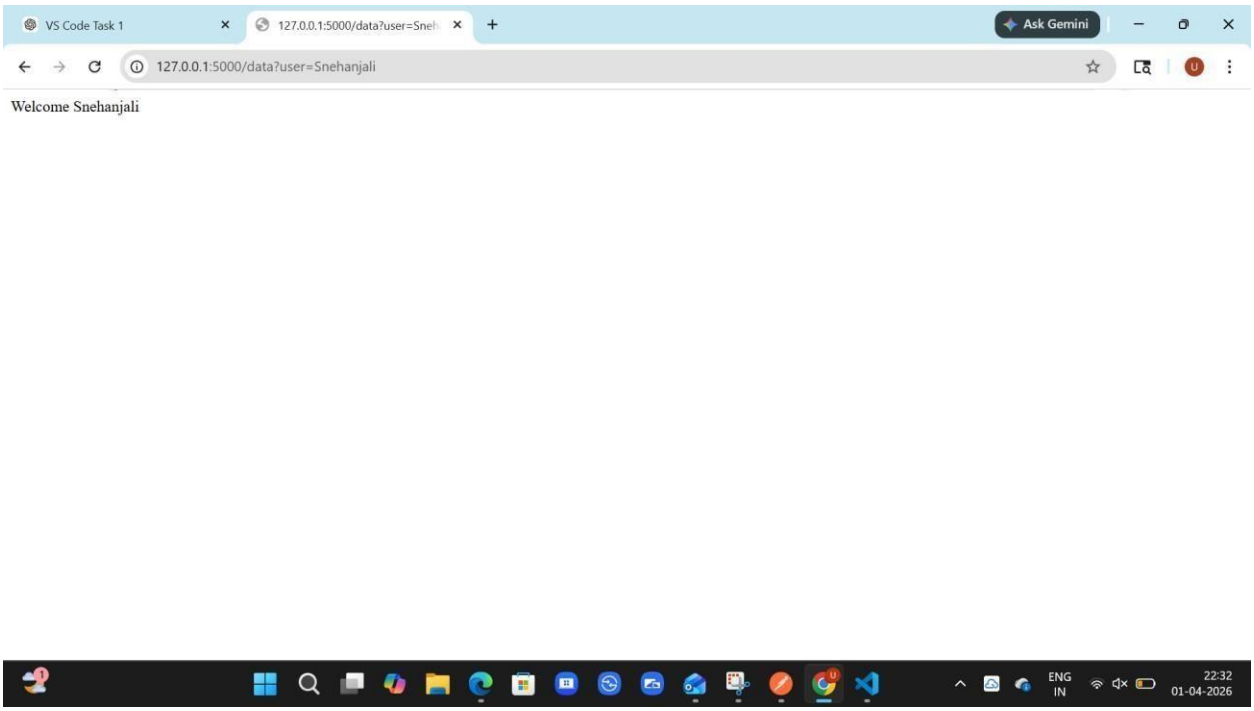
- The improved code restricts file types and allows only safe formats like `.txt`, `.png`, `.jpg`, and `.pdf`. Task3: API Security Review

Prompt:

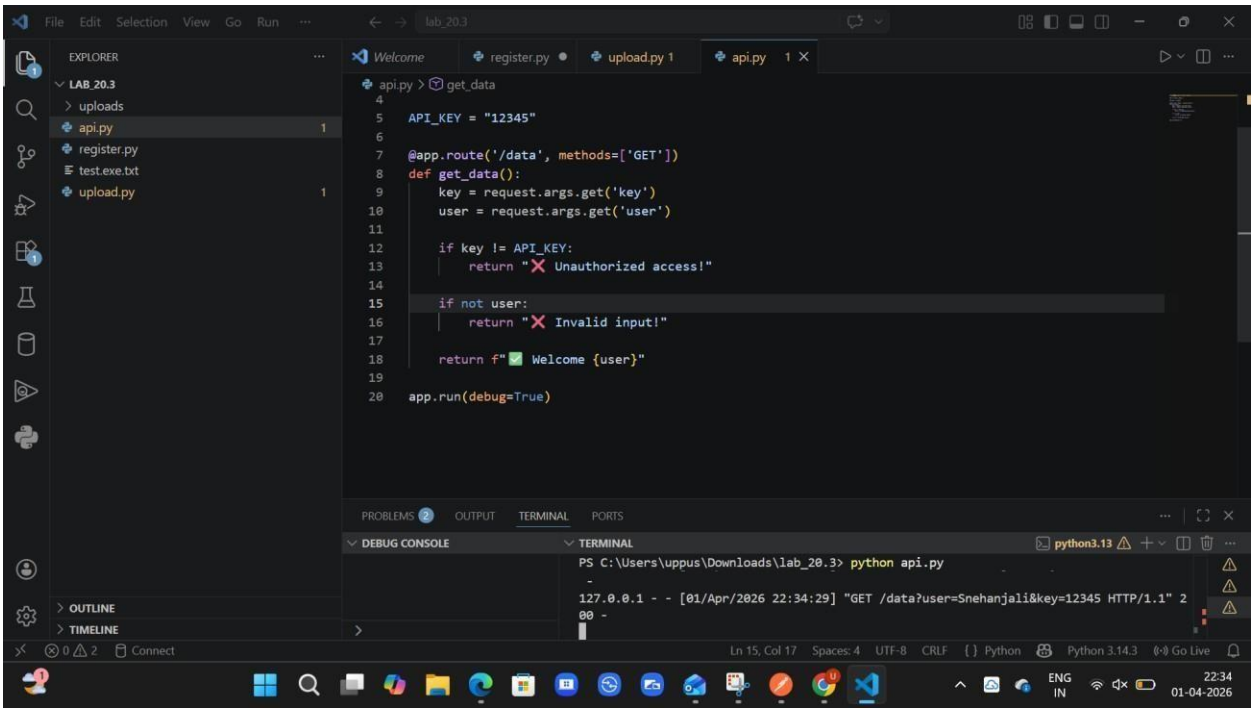
Generate a Python Flask API endpoint for user data Code:



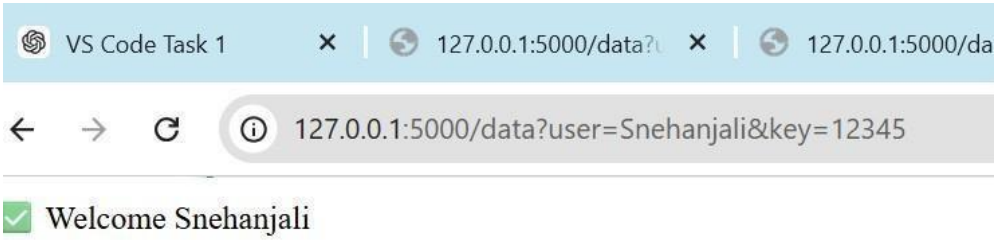
Output:



Corrected code:



Output:



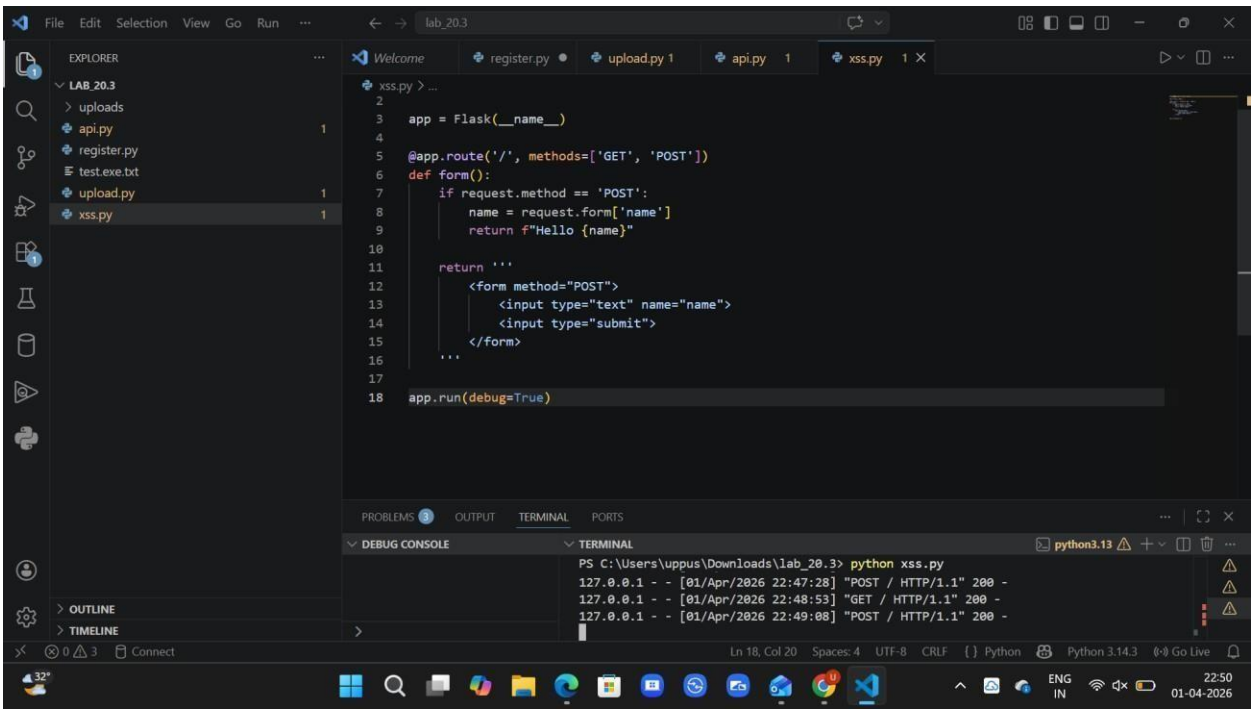
Explanation:

- The initial API does not have authentication, allowing anyone to access it
- It does not validate user input, making it insecure
- The improved API uses an API key and input validation for secure access

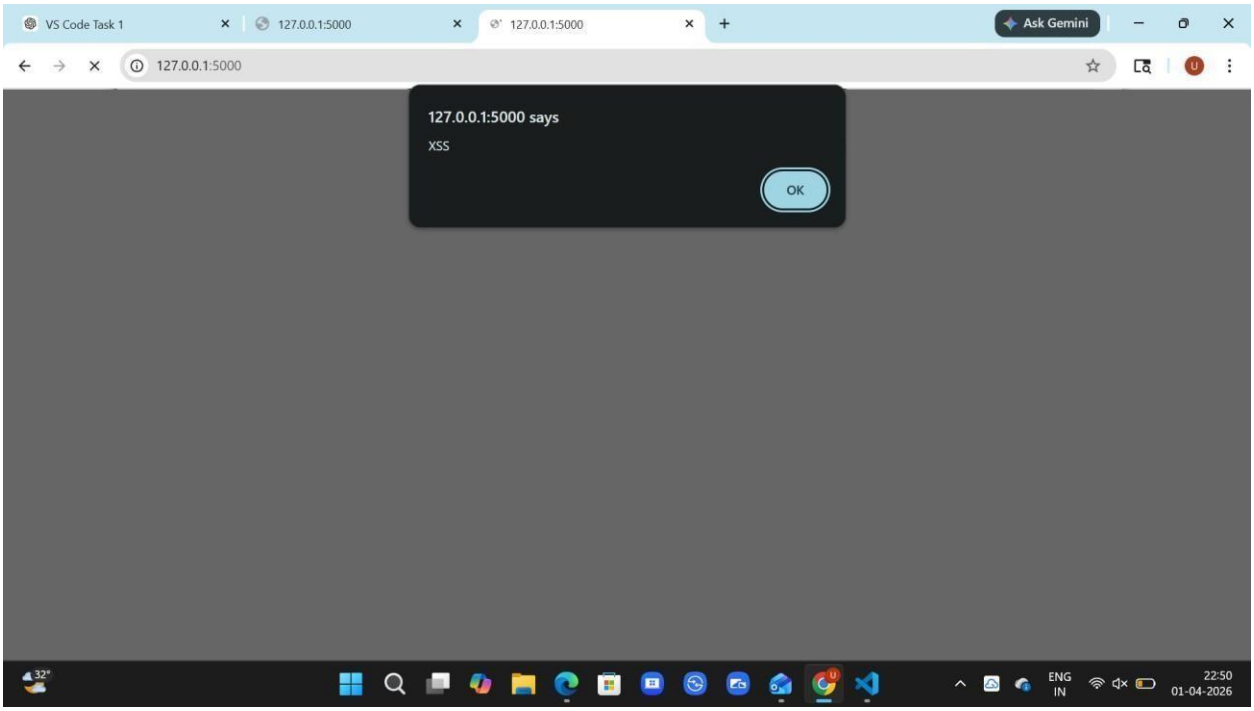
Task 4: Cross-Site Scripting (XSS) Check

Prompt:Generate a feedback form using HTML and JavaScript

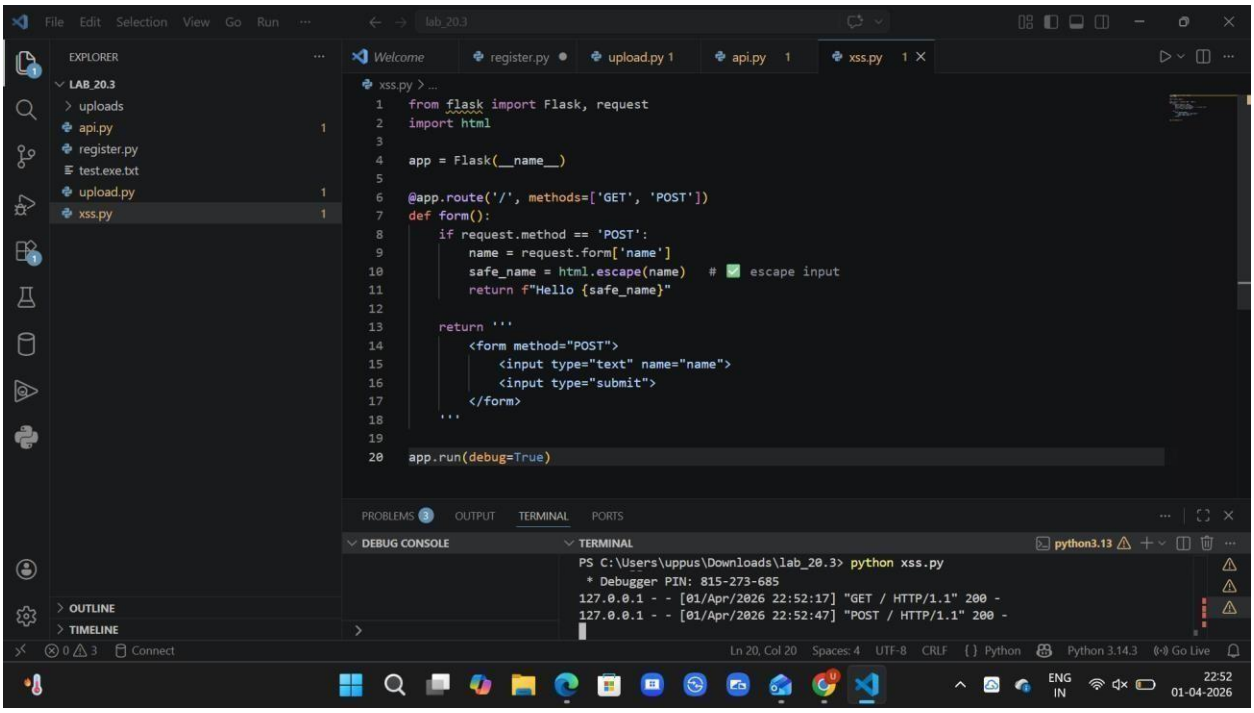
Code:



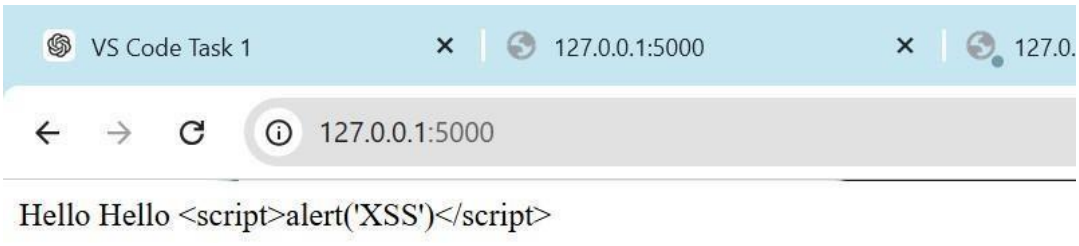
Output:



Corrected Code:



Output:



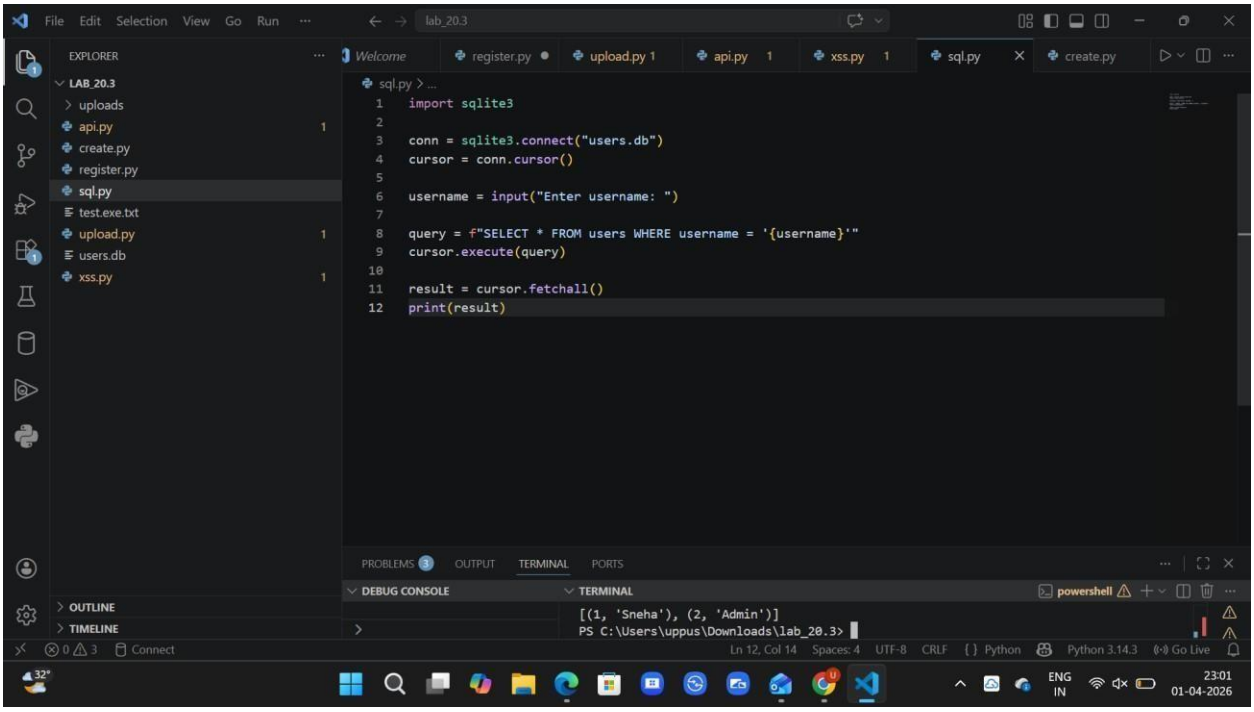
Explanation:

- The initial code displays user input directly without any validation, making it vulnerable to XSS attacks.
- Malicious scripts like `<script>alert('XSS')</script>` can execute in the browser.
- The secure code uses input escaping to prevent script execution and ensures safe output.

Task 5: SQL Injection Prevention

Prompt: Generate a Python script using SQLite to fetch user details from a database.

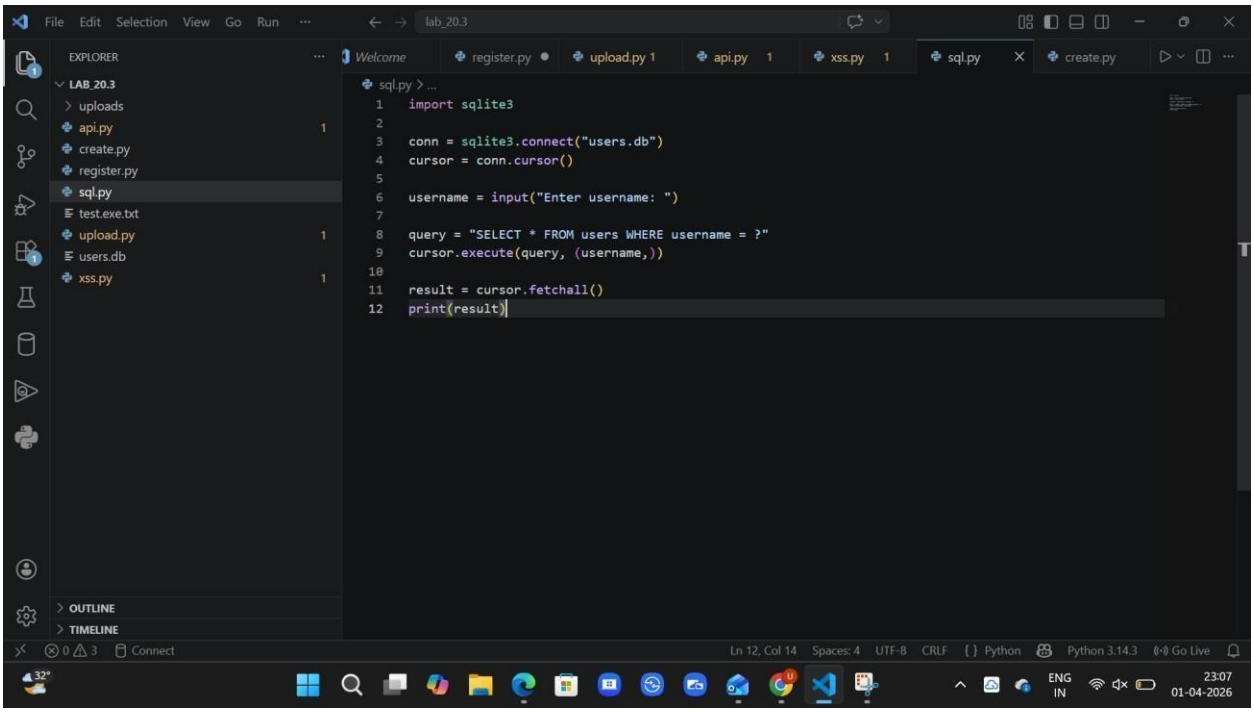
Code:



Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[(1, 'Sneha'), (2, 'Admin')]
PS C:\Users\uppus\Downloads\lab_20.3>
```

Corrected code:



Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[]
PS C:\Users\uppus\Downloads\lab_20.3>
```

Explanation:

- The secure code uses parameterized queries (`?`) instead of string concatenation.
- User input is treated as data, not as part of the SQL query.
- This prevents SQL Injection and ensures safe database operations.